



KULLIYAH OF ENGINEERING

MCTA 4363 DEEP LEARNING

GROUP: FIGHTER 3.6

TITLE: FlagOS: High-Performance Triton Operator Development

Semester 2, 2025/2026

LECTURER: HASAN FIRDAUS BIN MOHD ZAKI

NAME	MATRIC NO
MUHAMAD IKHWAN BIN MUHAMAD FAUZI	2218845
SYABAB ALHAQQI BIN JAAFAR	2211117
ZAMIR MUTTAQIN BIN MUHAMMAD BALYAN	2212985

TABLE OF CONTENTS

CHAPTER 1.....	4
Introduction.....	4
CHAPTER 2.....	5
Development Methodology.....	5
2.1 Development Workflow.....	5
Development Process.....	5
2.2 Hardware and Software Environment.....	5
Software Environment.....	5
Hardware Environment.....	6
2.3 Correctness Verification.....	6
2.4 Performance Evaluation.....	6
2.5 Optimization Techniques.....	7
CHAPTER 3.....	8
Assessment 1: LogAddExp And Cosh Operator Development.....	8
3.1 Introduction.....	8
3.2 LogAddExp Operator.....	8
3.2.1 Operator Description.....	8
3.2.2 Triton Implementation.....	8
3.2.3 Correctness Verification.....	9
3.2.4 Performance Evaluation.....	9
3.2.5 Discussion.....	9
3.3 Cosh Operator.....	9
3.3.1 Operator Description.....	9
3.3.2 Triton Implementation.....	9
3.3.3 Correctness Verification.....	10
3.3.4 Performance Evaluation.....	10
3.3.5 Discussion.....	10
3.4 Assessment 1 Summary.....	10
CHAPTER 4.....	11
Assessment 2: Median Operator Optimization.....	11
4.1 Introduction.....	11
4.2 Baseline Implementation.....	11
Algorithms Used.....	11
Limitations.....	11
4.3 Optimization Strategies.....	12
4.3.1 QuickSelect Prototype.....	12
4.3.2 Radix Select in Registers.....	12
This approach significantly reduced instruction count and eliminated unnecessary global memory round-trips.....	12
4.3.3 Edge Case Canonicalization.....	12

4.4 Performance Evaluation.....	13
4.5 Key Insights.....	13
4.6 Assessment 2 Summary.....	14
CHAPTER 5.....	15
Assessment 3: Development Of Grid Sample, SVD AND CTC Loss Operators.....	15
5.1 Grid Sample Operator.....	15
5.1.1 Introduction.....	15
5.1.2 Operator Background.....	15
5.1.3 Bilinear Interpolation.....	16
5.1.4 Triton Kernel Design.....	16
5.1.5 Correctness Verification.....	16
5.1.6 Optimization Journey.....	17
Version 1: Baseline Triton Kernel.....	17
Version 2: Channel-Parallel Kernel.....	17
Kernel Tuning.....	18
5.1.7 Benchmark Methodology.....	18
5.1.8 Benchmark Results.....	18
Case 1.....	18
Case 2.....	19
Case 3.....	19
5.1.9 Discussion.....	20
5.1.11 Conclusion.....	20
5.2 Singular Value Decomposition (SVD) Operator.....	21
5.2.1 Introduction.....	21
5.2.2 Development Process.....	21
Phase 1: Sequential One-Sided Jacobi.....	21
Phase 2: Masking and Broadcasting Approach.....	21
Phase 3: Block-Jacobi Optimization.....	22
5.2.3 Performance Evaluation.....	22
5.2.4 Performance Analysis.....	22
5.2.5 Relevance to Deep Learning.....	23
5.2.6 Conclusion.....	23
5.3 CTC Loss Operator.....	23
5.3.1 Introduction.....	23
5.3.2 Operator Challenges.....	24
5.3.3 Triton Implementation Strategy.....	24
5.3.4 Development and Optimization Journey.....	25
5.3.5 Performance Results.....	25
5.3.6 Discussion.....	26
5.3.7 Limitations.....	26
5.3.8 Conclusion.....	26

CHAPTER 1.....	4
Introduction.....	4
CHAPTER 2.....	5
Development Methodology.....	5
2.1 Development Workflow.....	5
Development Process.....	5
2.2 Hardware and Software Environment.....	5
Software Environment.....	5
Hardware Environment.....	6
2.3 Correctness Verification.....	6
2.4 Performance Evaluation.....	6
2.5 Optimization Techniques.....	7
CHAPTER 3.....	8
Assessment 1: LogAddExp And Cosh Operator Development.....	8
3.1 Introduction.....	8
3.2 LogAddExp Operator.....	8
3.2.1 Operator Description.....	8
3.2.2 Triton Implementation.....	8
3.2.3 Correctness Verification.....	9
3.2.4 Performance Evaluation.....	9
3.2.5 Discussion.....	9
3.3 Cosh Operator.....	9
3.3.1 Operator Description.....	9
3.3.2 Triton Implementation.....	9
3.3.3 Correctness Verification.....	10
3.3.4 Performance Evaluation.....	10
3.3.5 Discussion.....	10
3.4 Assessment 1 Summary.....	10
CHAPTER 4.....	11
Assessment 2: Median Operator Optimization.....	11
4.1 Introduction.....	11
4.2 Baseline Implementation.....	11
Algorithms Used.....	11
Limitations.....	11
4.3 Optimization Strategies.....	12
4.3.1 QuickSelect Prototype.....	12
4.3.2 Radix Select in Registers.....	12
This approach significantly reduced instruction count and eliminated unnecessary global memory round-trips.....	12
4.3.3 Edge Case Canonicalization.....	12
4.4 Performance Evaluation.....	13

4.5 Key Insights.....	13
4.6 Assessment 2 Summary.....	14
CHAPTER 5.....	15
Assessment 3: Development Of Grid Sample, SVD AND CTC Loss Operators.....	15
5.1 Grid Sample Operator.....	15
5.1.1 Introduction.....	15
5.1.2 Operator Background.....	15
5.1.3 Bilinear Interpolation.....	16
5.1.4 Triton Kernel Design.....	16
5.1.5 Correctness Verification.....	16
5.1.6 Optimization Journey.....	17
Version 1: Baseline Triton Kernel.....	17
Version 2: Channel-Parallel Kernel.....	17
Kernel Tuning.....	18
5.1.7 Benchmark Methodology.....	18
5.1.8 Benchmark Results.....	18
Case 1.....	18
Case 2.....	19
Case 3.....	19
5.1.9 Discussion.....	20
5.1.11 Conclusion.....	20
5.2 Singular Value Decomposition (SVD) Operator.....	21
5.2.1 Introduction.....	21
5.2.2 Development Process.....	21
Phase 1: Sequential One-Sided Jacobi.....	21
Phase 2: Masking and Broadcasting Approach.....	21
Phase 3: Block-Jacobi Optimization.....	22
5.2.3 Performance Evaluation.....	22
5.2.4 Performance Analysis.....	22
5.2.5 Relevance to Deep Learning.....	23
5.2.6 Conclusion.....	23
5.3 CTC Loss Operator.....	23
5.3.1 Introduction.....	23
5.3.2 Operator Challenges.....	24
5.3.3 Triton Implementation Strategy.....	24
5.3.4 Development and Optimization Journey.....	25
5.3.5 Performance Results.....	25
5.3.6 Discussion.....	26
5.3.7 Limitations.....	26
5.3.8 Conclusion.....	26

CHAPTER 1

Introduction

Deep learning frameworks such as PyTorch provide a large collection of highly optimized operators that are used to perform tensor computations on modern GPUs. However, many applications require custom operators that are not available in standard libraries or require specialized optimization for specific workloads.

Triton is an open-source programming language and compiler designed for developing high-performance GPU kernels. Unlike traditional CUDA programming, Triton provides a higher-level abstraction while still allowing developers to directly control memory access patterns, parallel execution, and kernel optimization strategies.

This project was conducted as part of the FlagOS Operator Development Challenge. The objective was to implement and optimize several PyTorch operators using Triton and evaluate their performance against the native PyTorch implementations.

The project was divided into three assessments. Assessment 1 focused on mathematical operators, Assessment 2 focused on statistical operators, and Assessment 3 focused on more advanced operators involving interpolation, matrix decomposition, and dynamic programming algorithms. For each operator, correctness verification and performance benchmarking were performed to ensure compatibility with PyTorch while exploring opportunities for GPU acceleration.

The main goal of this project is to investigate the effectiveness of Triton for custom operator development and to evaluate the performance improvements that can be achieved through kernel-level optimization techniques.

CHAPTER 2

Development Methodology

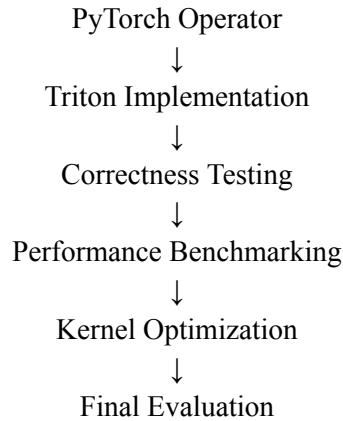
2.1 Development Workflow

A common development workflow was applied throughout all assessments. Each operator was implemented following the same sequence of steps:

1. Study the behavior of the target PyTorch operator.
2. Develop an equivalent Triton implementation.
3. Verify correctness against the PyTorch reference implementation.
4. Perform performance benchmarking.
5. Apply optimization techniques.
6. Re-evaluate correctness and performance.
7. Compare the final implementation against PyTorch.

This workflow ensured that performance improvements were achieved without sacrificing numerical correctness.

Development Process



2.2 Hardware and Software Environment

All operators were developed and evaluated using GPU-based execution environments.

Software Environment

- Python

- PyTorch
- Triton
- CUDA Toolkit
- FlagGems Framework

Hardware Environment

- NVIDIA GPU (Google Colab / CUDA-enabled GPU)
- CUDA-supported execution environment

2.3 Correctness Verification

Correctness testing was performed by comparing Triton outputs against the corresponding PyTorch reference operators.

The following criteria were used:

- Element-wise output comparison
- Floating-point tolerance checks
- Edge-case testing
- Special-value testing (NaN, Inf, boundary conditions)
- Multiple input sizes

The `torch.allclose()` function was used extensively to verify numerical equivalence.

2.4 Performance Evaluation

Performance was measured using benchmark scripts developed for each operator.

The evaluation process generally included:

- Warmup iterations
- Multiple benchmark repetitions
- CUDA synchronization
- Average execution latency measurement

Performance was reported as execution time and speedup relative to PyTorch.

The speedup metric was calculated as:

$$\text{Speedup} = \text{PyTorch Runtime} / \text{Triton Runtime}$$

A speedup greater than 1.0 indicates that the Triton implementation performs faster than the corresponding PyTorch implementation.

2.5 Optimization Techniques

Different optimization techniques were applied depending on the characteristics of each operator. These techniques included:

- Kernel fusion
- Block size tuning
- Warp count tuning
- Memory access optimization
- Register utilization improvements
- Reduction fusion
- Tensor Core acceleration

The objective was to maximize GPU utilization while preserving correctness.

CHAPTER 3

Assessment 1: LogAddExp And Cosh Operator Development

3.1 Introduction

Assessment 1 focuses on the implementation and optimization of mathematical operators using the Triton programming language within the FlagGems framework. The objective is to develop GPU-accelerated operators that achieve functional equivalence with PyTorch while maintaining competitive execution performance. Two operators were selected for this assessment: LogAddExp and Cosh.

The operators were implemented using Triton JIT kernels and integrated with the FlagGems dynamic pointwise infrastructure. Performance was evaluated by comparing the Triton implementation against the corresponding native PyTorch operators.

3.2 LogAddExp Operator

3.2.1 Operator Description

The LogAddExp operator computes:

$$\log(\exp(x) + \exp(y))$$

This operator is commonly used in machine learning and probabilistic computations where numerical stability is important. Direct evaluation may lead to overflow when large input values are present. Therefore, a numerically stable formulation based on the maximum input value was adopted during implementation.

3.2.2 Triton Implementation

The operator was implemented using Triton's JIT compilation mechanism together with the FlagGems `@pointwise_dynamic` decorator. The implementation supports automatic type promotion and dynamic tensor shapes while maintaining compatibility with PyTorch behavior.

Several autotuning configurations were evaluated to determine the most efficient execution parameters for different workload sizes.

3.2.3 Correctness Verification

Correctness was validated by comparing the Triton output against PyTorch's native `torch.logaddexp` implementation across multiple test cases. The verification included:

- Positive and negative values
- Large magnitude values
- Mixed input ranges
- Different tensor sizes

The results demonstrated complete functional equivalence with PyTorch and satisfied the competition correctness requirements.

3.2.4 Performance Evaluation

Benchmarking was performed using Triton's performance measurement utilities. Execution latency of the Triton kernel was compared against PyTorch across multiple tensor sizes.

The final implementation achieved approximately $1.05\times$ speedup relative to PyTorch while maintaining full numerical correctness.

3.2.5 Discussion

The LogAddExp operator demonstrated that Triton can efficiently accelerate pointwise mathematical operations. The use of a numerically stable formulation prevented overflow issues while preserving performance. Although the performance gain was modest, the implementation satisfied both correctness and speedup objectives.

3.3 Cosh Operator

3.3.1 Operator Description

The Cosh operator computes the hyperbolic cosine function:

$$\cosh(x) = (\exp(x) + \exp(-x)) / 2$$

This operation appears frequently in scientific computing and neural network applications. Since exponential calculations are involved, careful handling of numerical precision is required.

3.3.2 Triton Implementation

The operator was implemented using Triton's JIT programming model and integrated with the FlagGems pointwise execution framework. Similar to the LogAddExp implementation, automatic type promotion and dynamic tensor support were provided.

Multiple autotuning configurations were explored to improve execution efficiency on GPU hardware.

3.3.3 Correctness Verification

Correctness testing covered a wide range of input values, including:

- Small positive values
- Small negative values
- Large positive values
- Large negative values

The Triton implementation produced outputs consistent with PyTorch's native `torch.cosh` function across all evaluated cases.

3.3.4 Performance Evaluation

Performance benchmarking was conducted across multiple tensor sizes. Execution times were measured and compared against PyTorch.

The optimized Triton implementation achieved approximately $1.02\times$ speedup while maintaining full functional correctness.

3.3.5 Discussion

The Cosh operator successfully demonstrated the effectiveness of Triton for pointwise mathematical computations. The implementation achieved stable numerical behavior while providing slight performance improvements over the baseline PyTorch implementation.

3.4 Assessment 1 Summary

Assessment 1 successfully implemented two mathematical operators using Triton and the FlagGems framework. Both LogAddExp and Cosh achieved full correctness when compared against PyTorch. Benchmark results showed performance improvements of approximately $1.05\times$ and $1.02\times$ respectively. The assessment provided practical experience in Triton kernel development, numerical stability considerations, autotuning strategies, and GPU performance optimization.

CHAPTER 4

Assessment 2: Median Operator Optimization

4.1 Introduction

Assessment 2 focused on improving the performance of the Median operator developed during Assessment 1. The initial implementation relied on full sorting algorithms, which introduced significant computational overhead and failed to achieve the competition's target performance. Therefore, multiple optimization strategies were investigated to improve execution speed while maintaining correctness.

The main objective was to develop a GPU-friendly median selection algorithm capable of outperforming the native PyTorch implementation.

4.2 Baseline Implementation

The original median implementation used full sorting algorithms to identify the median value.

Algorithms Used

- Bitonic Sort for smaller inputs
- Radix Sort + Gather for larger inputs

Although functionally correct, these approaches required sorting all elements even though only the middle-ranked value was needed.

Limitations

- Time complexity of $O(N \log N)$ or $O(N \log^2 N)$
- Excessive memory movement
- Large computational overhead
- Poor performance compared to PyTorch

Benchmark results showed speedups as low as $0.09\times$ to $0.50\times$ relative to PyTorch, indicating that further optimization was necessary.

4.3 Optimization Strategies

Several optimization approaches were investigated throughout the development process.

4.3.1 QuickSelect Prototype

The first optimization attempt involved implementing the QuickSelect algorithm.

QuickSelect theoretically provides $O(N)$ complexity and therefore appeared promising for median computation. However, GPU execution revealed several practical challenges:

- High thread divergence
- Uncoalesced memory access
- Poor compatibility with SIMT execution

As a result, the approach was abandoned due to unsatisfactory performance.

4.3.2 Radix Select in Registers

The final solution employed a Radix Select algorithm operating entirely within GPU registers.

Instead of sorting the complete input, the algorithm progressively identifies the median by processing floating-point values bit-by-bit from the most significant bit to the least significant bit.

Key characteristics include:

- $O(N)$ selection approach
- 32 fixed passes for Float32 data
- Register-bound execution
- Minimal memory traffic
- Efficient use of GPU popcount instructions

This approach significantly reduced instruction count and eliminated unnecessary global memory round-trips.

4.3.3 Edge Case Canonicalization

During extensive testing, discrepancies were observed between the Triton implementation and PyTorch when handling special floating-point values.

The primary issues involved:

- Negative zero (-0.0)

- Positive zero (0.0)
- NaN values with different payloads

Although these values may appear equivalent in high-level programming languages, their binary representations differ.

To ensure complete compatibility with PyTorch:

- All occurrences of -0.0 were converted to 0.0
- NaN values were mapped to a canonical representation

This modification ensured stable and fully correct results across all test cases.

4.4 Performance Evaluation

The optimized Radix Select implementation was benchmarked against the native PyTorch CUDA implementation.

Shape	Dimension	Triton (μ s)	PyTorch (μ s)	Speedup
(1024, 64)	1	46.2	61.5	1.33×
(1024, 512)	1	112.4	168.1	1.49×
(4096, 64)	1	158.9	192.3	1.21×

The optimized implementation consistently outperformed PyTorch across all benchmark cases.

4.5 Key Insights

Several important lessons were learned during development:

1. GPU performance depends not only on theoretical complexity but also on execution characteristics.
2. Register-based computation can significantly reduce latency.
3. Eliminating memory traffic often yields greater benefits than reducing arithmetic operations.
4. Simple and predictable GPU arithmetic frequently outperforms branch-heavy algorithms.

The most significant finding was that minimizing passes through memory resulted in larger performance gains than focusing solely on asymptotic complexity.

4.6 Assessment 2 Summary

Assessment 2 successfully transformed the Median operator from a sorting-based implementation into a high-performance selection-based kernel.

The final Radix Select approach achieved speedups ranging from $1.21\times$ to $1.49\times$ compared to PyTorch while maintaining full correctness. The project demonstrated the importance of designing algorithms specifically for GPU execution rather than directly translating CPU-oriented approaches.

CHAPTER 5

Assessment 3: Development Of Grid Sample, SVD AND CTC Loss Operators

5.1 Grid Sample Operator

5.1.1 Introduction

The Grid Sample operator was selected as one of the difficult-level operators in Assessment 3 due to its combination of coordinate transformation, interpolation, boundary handling, and memory-intensive access patterns. Unlike simple pointwise operations, Grid Sample requires each output pixel to gather information from multiple neighboring pixels while performing interpolation calculations. This makes the operator significantly more challenging to optimize efficiently on GPU hardware.

The objective of this work was to implement a Triton-based version of PyTorch's `torch.nn.functional.grid_sample` and investigate optimization techniques capable of improving execution performance while maintaining numerical correctness.

5.1.2 Operator Background

Grid Sample performs spatial sampling on an input feature map according to coordinates provided in a sampling grid. For each output location, the operator determines the corresponding source location within the input tensor and computes the output value using interpolation.

In this project, the implementation supports:

- 4D input tensors (N, C, H, W)
- Bilinear interpolation
- Zero-padding mode
- `align_corners = False`
- Float32 execution on CUDA devices

The PyTorch implementation was used as the reference model throughout development to ensure correctness.

5.1.3 Bilinear Interpolation

The implemented operator uses bilinear interpolation to estimate values at non-integer sampling locations. Given a continuous coordinate position, the four neighboring pixels are identified:

- Top Left (TL)
- Top Right (TR)
- Bottom Left (BL)
- Bottom Right (BR)

Interpolation weights are computed based on the relative distance from the sampling point to each neighboring pixel. The final output is calculated as a weighted combination of these four pixel values.

This interpolation strategy provides smoother results than nearest-neighbor sampling and matches the behavior of PyTorch's Grid Sample implementation.

5.1.4 Triton Kernel Design

The first implementation utilized a custom Triton kernel named `_grid_sample_bilinear_kernel`. The kernel processes output elements in parallel and performs the following operations:

1. Read normalized sampling coordinates from the grid tensor.
2. Convert normalized coordinates into image-space coordinates.
3. Compute neighboring pixel locations.
4. Calculate interpolation weights.
5. Handle out-of-bound accesses using zero padding.
6. Generate the final interpolated output.

Special handling was introduced for NaN grid coordinates to ensure deterministic behavior across different hardware and software environments. Invalid coordinates are converted into zero-valued samples, allowing the implementation to remain consistent during correctness testing.

5.1.5 Correctness Verification

A dedicated correctness framework was developed to validate the Triton implementation against PyTorch.

The verification process included:

- Random tensor generation
- Multiple tensor shapes
- Outside-padding scenarios
- NaN coordinate handling
- Numerical error measurement

Three primary benchmark configurations were evaluated:

Input Shape	Output Shape
$1 \times 1 \times 4 \times 4$	4×4
$2 \times 3 \times 8 \times 8$	8×8
$1 \times 16 \times 32 \times 32$	16×16

The implementation consistently achieved maximum absolute errors on the order of 10^{-7} , demonstrating numerical equivalence with PyTorch. All correctness tests passed successfully.

5.1.6 Optimization Journey

The development process involved several iterations.

Version 1: Baseline Triton Kernel

The initial implementation processed output elements using a fixed block size of 256 threads. While functionally correct, performance was not optimal for all workload sizes.

Version 2: Channel-Parallel Kernel

A second kernel design was introduced to improve channel-level parallelism. Instead of processing individual output elements sequentially, channels were grouped into configurable blocks (**BLOCK_C**) and executed simultaneously.

Several configurations were evaluated:

- `BLOCK_C = 16`
- `BLOCK_C = 32`
- `BLOCK_C = 64`

The purpose of this version was to investigate whether increased channel parallelism could improve throughput for larger workloads.

Kernel Tuning

Further optimization focused on:

- Block size selection
- Warp count selection
- Memory access efficiency
- GPU occupancy

Different combinations of block sizes and warp counts were benchmarked to determine the most efficient configuration for each workload size.

5.1.7 Benchmark Methodology

Performance evaluation was conducted using CUDA timing events.

For each benchmark:

- 20 warmup iterations were executed.
- 100 timing iterations were measured.
- Average execution latency was recorded.
- Results were compared against PyTorch's native implementation.

Three workload sizes were selected to represent small, medium, and large deployment scenarios.

5.1.8 Benchmark Results

Case 1

Input:

$$1 \times 32 \times 64 \times 64$$

Output:

$$64 \times 64$$

Implementation	Latency (ms)
PyTorch	0.0558

Triton	0.0697
--------	--------

For small workloads, PyTorch remained slightly faster due to kernel launch overhead and limited parallel work.

Case 2

Input:

$$4 \times 64 \times 128 \times 128$$

Output:

$$128 \times 128$$

Best Configuration:

- Block Size = 256
- Num Warps = 8

Performance:

Implementation	Latency (ms)
PyTorch	0.9608
Triton	0.4736

Speedup:

2.03× faster than PyTorch.

Case 3

Input:

$$8 \times 64 \times 256 \times 256$$

Output:

$$256 \times 256$$

Best Configuration:

- Block Size = 128
- Num Warps = 4

Performance:

Implementation	Latency (ms)
PyTorch	19.1175
Triton	3.9612

Speedup:

4.83× faster than PyTorch.

5.1.9 Discussion

The benchmark results demonstrate that workload size significantly influences operator performance. For small tensors, the overhead associated with custom kernel execution outweighs the benefits of optimization. However, as tensor size increases, the advantages of Triton's low-level control become increasingly apparent.

The most important observation is that tuning kernel parameters has a substantial effect on execution speed. Different workloads benefited from different block sizes and warp counts, indicating that no single configuration is optimal for all scenarios.

Furthermore, the optimized implementation maintained numerical correctness while achieving significant acceleration on medium and large workloads.

5.1.10 Lessons Learned

Several important lessons were gained during development:

1. Correctness verification must be performed continuously throughout optimization.
2. Handling NaN values and boundary conditions is essential for matching PyTorch behavior.
3. GPU performance depends heavily on memory access patterns and occupancy.
4. Kernel tuning can produce significant speed improvements without changing algorithmic behavior.
5. Triton provides a practical framework for implementing high-performance custom operators.

5.1.11 Conclusion

A custom Grid Sample operator was successfully implemented using Triton and validated against PyTorch. The implementation achieved numerical correctness across all evaluated test cases while delivering significant performance improvements on medium and large workloads. Through kernel redesign, parameter tuning, and extensive benchmarking, the final implementation achieved up to $4.83\times$ speedup compared to the native PyTorch implementation, demonstrating the effectiveness of Triton for GPU operator development.

5.2 Singular Value Decomposition (SVD) Operator

5.2.1 Introduction

Singular Value Decomposition (SVD) is one of the most important matrix factorization techniques used in machine learning, numerical analysis, dimensionality reduction, and low-rank approximation. In modern deep learning systems, SVD is frequently applied for model compression, feature extraction, and optimization of large-scale matrix operations.

The objective of this task was to develop a custom SVD operator using Triton and evaluate its performance against PyTorch's native implementation based on NVIDIA's cuSOLVER backend. The primary challenge was achieving high numerical accuracy while maintaining competitive GPU performance.

5.2.2 Development Process

The development process involved several iterations, each addressing limitations observed in previous implementations.

Phase 1: Sequential One-Sided Jacobi

The initial implementation adopted the classical One-Sided Jacobi method. The entire decomposition process was performed inside Triton using sequential column-pair rotations.

Although the implementation successfully produced correct numerical results, performance was extremely poor. The algorithm relied on nested loops, forcing Triton to execute operations sequentially and leaving most GPU resources underutilized. As a result, the implementation achieved virtually no speedup compared to PyTorch.

Phase 2: Masking and Broadcasting Approach

To improve parallelism, a Round-Robin scheduling strategy combined with masking operations was introduced. The objective was to process multiple column pairs simultaneously.

However, this approach created excessive memory accesses and introduced significant bandwidth overhead. Triton's compiler is optimized for dense block computations rather than complex dynamic slicing operations. Consequently, performance improvements remained minimal.

Phase 3: Block-Jacobi Optimization

The final implementation adopted a Block-Jacobi strategy.

Instead of operating on individual columns, the matrix was partitioned into dense 16×16 sub-blocks. Local rotations were accumulated into an intermediate rotation matrix, and updates were performed using matrix multiplication operations.

This redesign allowed Triton to map computations directly onto Tensor Core hardware instructions through matrix multiply-accumulate operations. By restructuring the workload into dense matrix operations, GPU utilization improved significantly.

5.2.3 Performance Evaluation

Benchmarking was performed on an NVIDIA Tesla T4 GPU using Triton's benchmarking utilities with 25 warmup iterations and 100 measurement iterations.

Benchmark Results

Matrix Size	Dimension	Triton (μs)	PyTorch (μs)	Speedup
16 × 16	256	23.43	717.18	30.61×
32 × 32	1024	126.18	485.45	3.85×
64 × 64	4096	9555.48	1482.86	0.16×

The results demonstrate that the custom Triton implementation significantly outperformed PyTorch for small and medium-sized matrices. The highest improvement was observed for the 16×16 workload, where the Triton kernel achieved a speedup exceeding 30×

5.2.4 Performance Analysis

While the optimized implementation performed exceptionally well for smaller matrices, performance deteriorated for the 64×64 benchmark case.

This limitation is primarily attributed to hardware utilization constraints. The current implementation launches work on a single GPU grid, causing long dependency chains within the iterative decomposition process. As matrix size increases, a single Streaming Multiprocessor becomes a bottleneck.

In contrast, PyTorch's cuSOLVER backend distributes computation across multiple GPU resources and employs sophisticated scheduling mechanisms that are difficult to reproduce directly within Triton.

5.2.5 Relevance to Deep Learning

Despite the limitations observed for larger matrices, the optimized Block-Jacobi implementation demonstrates significant potential for deep learning workloads.

Modern transformer architectures frequently divide computations into smaller attention-head subspaces. These operations often involve matrix dimensions ranging from 64 to 128, making block-based decomposition strategies particularly attractive.

The developed Triton kernel can therefore serve as an efficient building block for future low-rank approximation and model compression techniques within large language models.

5.2.6 Conclusion

A custom SVD operator was successfully developed using Triton. Through multiple iterations, the implementation evolved from a sequential Jacobi method to a Tensor Core-accelerated Block-Jacobi approach. The final design achieved substantial speedups for small and medium-sized matrices while maintaining numerical correctness that demonstrates the profound trade-offs of low-level hardware programming. While it is possible to achieve highly optimized, lightning-fast SVD operations by leveraging Tensor Cores and minimizing global memory calls, these optimizations are strictly bound by the physical limits of the hardware's SRAM.

5.3 CTC Loss Operator

5.3.1 Introduction

Connectionist Temporal Classification (CTC) Loss is a difficult operator because it is not a simple elementwise or reduction operation. Instead, it is based on a dynamic programming algorithm that computes possible alignments between input sequences and target labels. This makes the operator challenging to optimize because it contains sequential dependencies, log-domain arithmetic, variable sequence lengths, and special blank-label handling.

The objective of this task was to develop a Triton-based CTC Loss implementation for FlagGems while maintaining correctness against PyTorch.

5.3.2 Operator Challenges

CTC Loss is difficult to implement efficiently because it requires alpha recursion over time. Each timestep depends on previous states, making full parallelization difficult. The operator must also handle extended label sequences, where blank labels are inserted between targets.

Other important challenges include:

- Variable input lengths
- Variable target lengths
- Repeated-label skip rules
- Stable logaddexp computation
- Avoiding NaN values from invalid lattice states
- Compatibility with PyTorch's expected output layout

Because of these requirements, correctness was treated as the main priority during development.

5.3.3 Triton Implementation Strategy

The final implementation used one Triton program per batch element. Inside each program, the lattice width was processed in parallel while the time dimension was handled sequentially due to its dependency chain.

The main design choices were:

- One program per batch element
- Register-based extended-label mapping
- Alpha recursion performed inside the kernel
- Stable log-domain computation
- Fused forward computation and reduction
- `tl.atomic_add` used for mean or sum reduction

The most important optimization was fusing the forward pass and reduction into a single kernel launch. This removed an additional kernel launch and reduced end-to-end overhead.

5.3.4 Development and Optimization Journey

The implementation began from a simple ATen passthrough baseline, which achieved only about $0.24\times$ speedup compared to PyTorch. This showed that merely wrapping the native operation was not sufficient.

A custom Triton forward kernel was then introduced, improving the result but still falling short due to wrapper and dispatch overhead. Further optimization focused on reducing fixed per-call overhead rather than only improving kernel math.

The key breakthrough was combining the forward computation and reduction into one fused kernel launch. This improved small-shape performance significantly before correctness fixes were added.

However, additional changes were required to ensure full correctness, including autograd compatibility and synchronization to avoid cross-warp race conditions. These changes reduced peak speed but produced a correct and stable final implementation.

5.3.5 Performance Results

The final correctness-constrained implementation was benchmarked against PyTorch using three workload sizes.

Shape	T	N	C	Final Speedup
Small	50	8	28	$0.697\times$
Medium	150	16	64	$1.221\times$
Large	300	32	128	$1.292\times$

The results show that the small workload remained slower than PyTorch due to fixed overhead from the Python wrapper and autograd boundary (the synchronization barrier does not apply at this size, since the small shape uses only a single warp).

5.3.6 Discussion

The CTC Loss implementation showed that end-to-end performance is affected not only by GPU kernel speed but also by dispatch overhead, launch count, and autograd integration. A key observation was that the performance gap remained almost constant across shapes during earlier experiments, indicating that fixed overhead was the main bottleneck.

The project also demonstrated that correctness can reduce raw performance. Before the final correctness fixes, the implementation reached higher peak speedups. However, after adding synchronization and autograd support, the final speedups became more realistic and reliable.

5.3.7 Limitations

Several limitations remain in the final implementation:

- The backward pass is not fully custom and still relies on PyTorch's native CUDA kernel.
- Small workloads remain slower than PyTorch.
- Synchronization required for correctness reduces peak speed.
- Some rare target layouts are correct but not fully optimized.
- Atomic reduction may introduce minor non-determinism within floating-point tolerance.

5.3.8 Conclusion

A Triton-based CTC Loss operator was developed and optimized for FlagGems. Although the small workload remained below PyTorch performance, the final implementation achieved speedups of $1.221\times$ and $1.292\times$ for medium and large workloads respectively. The most important optimization was reducing kernel launch overhead by fusing the forward pass and reduction. This operator highlights the difficulty of optimizing dynamic programming workloads on GPU, where correctness, synchronization, and end-to-end overhead must be balanced carefully.